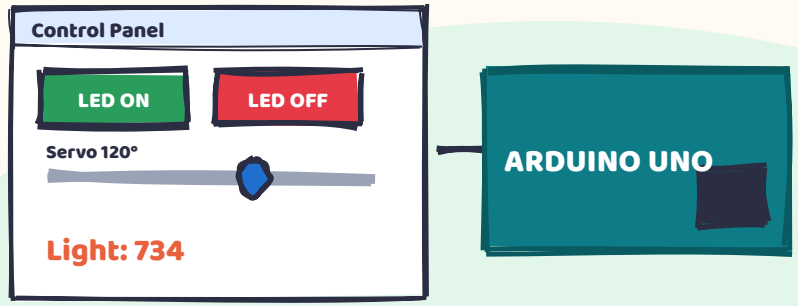
Young Innovators · Class 28

# Python Control Panel for Arduino

Click a button on screen. Something moves in the real world.

Protocol pyserial tkinter



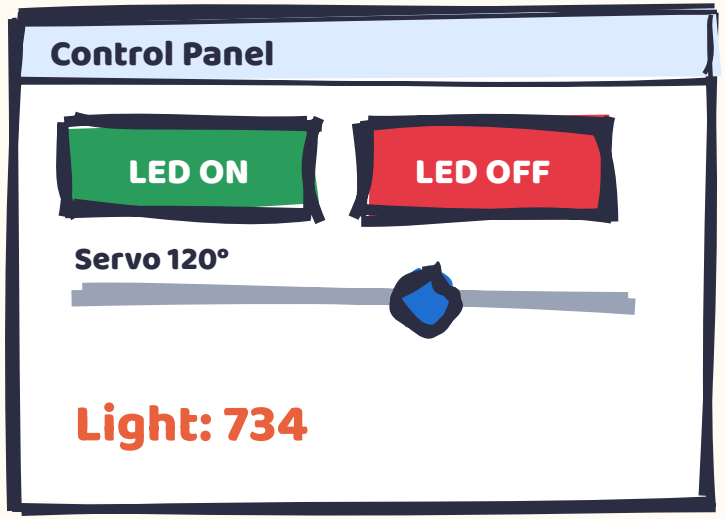

Did you know?
Factories, labs and even satellites are run from control panels like the one you'll build: buttons and live readings on a screen, hardware far away.

Let's think...

# Why type commands when you could click?

**Serial Monitor is boring**

So far we talk to the Arduino by typing in the Serial Monitor, or by re-uploading code. Real machines have **control panels**: buttons, sliders, gauges. Can Python give our Arduino a proper dashboard?




Type faster?

Buy a remote?

Build an app!

**Today:** you build a **Python app** with buttons and a slider that controls an LED and a servo, and shows a live sensor reading.

LEARN

Learning objectives

# By the end of this class I can...

| | | |
|---|---|---|
| 1 | **Design a protocol:** design simple **text commands** (like L1, S90) that the Arduino understands | **Proof:** I wrote a command table for my project |
| 2 | **Python ↔ Arduino:** send commands and read replies with **pyserial** | **Proof:** my Python script turns the LED on and reads the light level |
| 3 | **GUI control panel:** build a **tkinter** window with buttons, a slider and a live reading | **Proof:** I control my hardware by clicking, not typing |

LEARN

Objective 2 · Python ↔ Arduino · video

# Video time!




**The inner workings of Serial Communication Explained**

Programming Electronics Academy · about 5 minutes · youtu.be/qBfa-onviDA

**Watch for...**

1. How are bits sent **one at a time** down a wire?
2. What is a **baud rate**?
3. Why must both sides use the **same** settings?

Scan to watch

**After the video:** What would happen if Python sent at 9600 baud but the Arduino listened at 115200?

Objective 1 · Design a protocol

# A protocol is an agreement

## The idea

Python and Arduino must agree on **what messages mean**. Keep it simple: one letter for the action, then a number, then a newline **\n**:

**L1** = LED on, **L0** = LED off
**S90** = servo to 90°
**R** = read the light sensor → Arduino replies **734**

The newline marks the end, so the Arduino knows the message is complete.

## Example: a restaurant order code

| | |
|---|---|
| **L1** | → **LED on** |
| **L0** | → **LED off** |
| **S90** | → **servo 90°** |
| **R** | → **read light** |

A waiter writes 'T4 2xB 1xC' on a slip: table 4, two biryanis, one chai. Short codes, agreed by everyone in the kitchen, never confused.

Objective 2 · Python ↔ Arduino

# pyserial: Python's USB cable

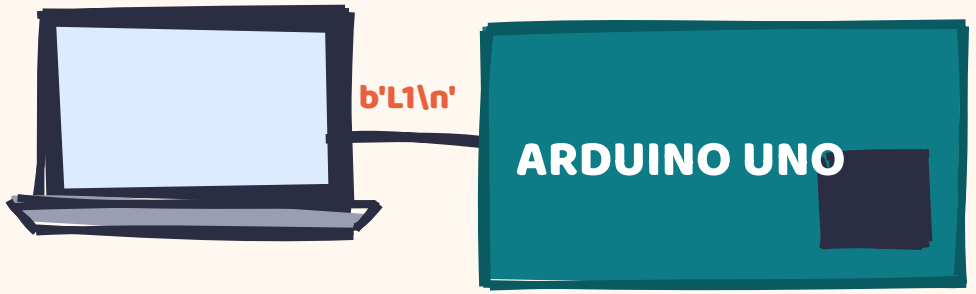
## The idea

Install **pyserial** (Tools → Manage packages). Then:

**ser = serial.Serial('COM3', 9600, timeout=1)**
(Mac/Linux: '/dev/ttyUSB0' or similar)
**ser.write(b'L1\n')** sends bytes
**ser.readline()** reads one reply line

Opening the port **resets** the Arduino, so wait ~2 s before sending.
And only one program can use the port: close Serial Monitor!

## Example: a phone call




Dial the number (open the port), wait for 'hello' (2 s reset), talk (write), listen (readline), hang up (close). Two people can't both use one phone line.

Objective 3 · GUI control panel

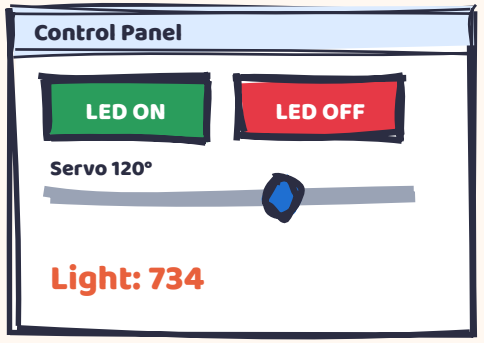
# GUIs are event-driven

## The idea

A **tkinter** app doesn't run top-to-bottom. It builds the window, then waits in **mainloop()** for **events**:
click a Button → its **command** function runs
drag a Scale (slider) → its function gets the new value

For live readings, **root.after(500, update)** asks tkinter to run update() again in 500 ms, forever, without freezing the window.

## Example: a shop assistant




A shop assistant doesn't follow a script. They wait, and react to whatever customer arrives. Your GUI waits and reacts to whatever the user clicks.

LEARN

Real world

# Control panels everywhere



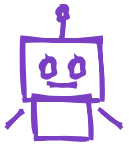
**Power stations**
SCADA screens control turbines and switches



**Mission control**
Engineers send commands to spacecraft



**Smart greenhouses**
Click to open vents, view temperature



**Lab robots**
Scientists run experiments from a laptop

? Think of one machine at school you would like to control from a laptop. What buttons would it need?

**Tiny protocols, huge systems**
Many industrial machines still use short text commands over serial, just like ours.

Quick quiz

# Protocol check

| Question | Answer |
| --- | --- |
| 1. What does S45 mean in our protocol? | |
| 2. Why end every command with \n? | |
| 3. Why wait 2 s after opening the port? | |
| 4. Which function do buttons call? | |
| 5. Why use root.after() instead of a while loop? | |

Answers

Cover! 1: servo to 45° · 2: marks the end of the message · 3: opening the port resets the Arduino · 4: their command= function · 5: a while loop freezes the window

Part 2 · Make it

# Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Human protocol
2. Our control kit
3. Arduino: the command listener
4. Python: the control panel
5. Make it your own panel
6. Panel tests
7. Control engineer challenges

Class game

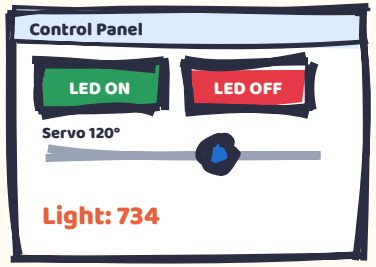
# Human protocol

## How to play

1. Pairs: one is **Python** (writes commands on cards), one is **Arduino** (acts them out).
2. Agree a protocol first: e.g. H1 = hand up, H0 = hand down, T90 = turn 90°, R = say a number 0 to 1023.
3. Python sends 10 cards fast. Arduino must act exactly, or reply 'ERR'.
4. Swap. Then try a broken card like 'T9O'. What should happen?

## Then discuss

What made a good protocol: short, clear, or both? How should the Arduino handle nonsense?




Big idea
Clear rules on both sides = no confusion.

MAKE

Build it

# Our control kit

**Arduino Uno**
Runs the command listener

**LED + 220 Ω**
Pin 13 (or built-in LED)

**SG90 servo**
Signal on pin 9

**LDR + 10 kΩ**
Light reading on A0

**Thonny + pyserial**
Python side

**Command table**
Your protocol on paper

Find your port: Arduino IDE → Tools → Port (e.g. COM3, /dev/cu.usbmodem...).

One at a time
Close the Arduino Serial Monitor before running Python, or you'll get 'Access denied / port busy'.

MAKE

Objective 1 · Design a protocol · Build step 1 of 3

# Arduino: the command listener

```ino
#include <Servo.h>
Servo sv;

void setup() {
  Serial.begin(9600);
  pinMode(13, OUTPUT);
  sv.attach(9);
}

void loop() {
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();
    char c = cmd.charAt(0);
    int n = cmd.substring(1).toInt();

    if (c == 'L')      digitalWrite(13, n == 1);
    else if (c == 'S') sv.write(constrain(n, 0, 180));
    else if (c == 'R') Serial.println(analogRead(A0));
    else               Serial.println("ERR");
  }
}
```

**Test first**

Upload, open Serial Monitor (Newline ending), type L1, S120, R. Does it work?

**constrain**

Protects the servo from silly values like S999.

**ERR**

Unknown commands get a polite error, not chaos.

Objective 3 · GUI control panel · Build step 2 of 3

# Python: the control panel

```
import serial, time, tkinter as tk

ser = serial.Serial("COM3", 9600, timeout=1)   # your port!
time.sleep(2)                                  # Arduino resets

def send(cmd):
    ser.write((cmd + "\n").encode())

def update():
    send("R")
    reply = ser.readline().decode().strip()
    light.config(text="Light: " + reply)
    root.after(500, update)                    # again in 0.5 s

root = tk.Tk()
root.title("Control Panel")
tk.Button(root, text="LED ON",  bg="green", command=lambda: send("L1")).pack(fill="x")
tk.Button(root, text="LED OFF", bg="red",   command=lambda: send("L0")).pack(fill="x")
tk.Scale(root, from_=0, to=180, orient="horizontal", label="Servo",
         command=lambda v: send("S" + v)).pack(fill="x")
light = tk.Label(root, text="Light: ?", font=("Arial", 24))
light.pack()
update()
root.mainloop()
```

**lambda**

A tiny function made on the spot, so each button sends its own command.

**after()**

Repeats update() every 0.5 s without freezing the window.

**encode()**

Serial sends bytes, so we turn the string into bytes.

MAKE

Objective 3 · Build step 3 of 3

# Make it your own panel

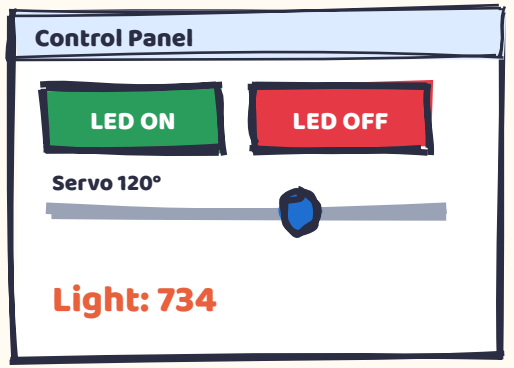
**a** Add a **Blink** button: new command B5 = blink 5 times. Update BOTH sides.

**b** Show the light level as a **bar** (a tk.Canvas rectangle that grows).

**c** Add an **automatic mode** checkbox: if dark, Python sends L1 by itself.

**d** Write your full **command table** in a comment at the top of both programs.

**e** Swap laptops: can another group use your panel without any help?




**Check:** A classmate can control your hardware using only your panel.

Test like an engineer

# Panel tests

| Test | Works? | Notes |
|---|---|---|
| LED ON / OFF buttons | | |
| Servo slider 0 to 180 | | |
| Light reading updates live | | |
| Bad command 'X9' → ERR | | |
| New command added | | |

## Debug checklist

SerialException / port busy? Close Serial Monitor, check the port name.
Nothing happens on first click? You didn't wait 2 s after opening.
Label shows blank? readline() timed out: is the Arduino replying to R?

Test the Arduino side alone first (Serial Monitor), then Python. Find bugs in one half at a time.

MAKE

Level up!

# Control engineer challenges

## Choose your challenge



**Graph:** plot the light level live with matplotlib inside the window.



**Safety:** add an EMERGENCY STOP button that also works if you press Space.



**Checksum:** add a simple checksum so corrupted commands are rejected.



**Wireless:** use an ESP32 and send the same commands over Wi-Fi.

## Think like an engineer

Why is it a good idea for the Arduino to reply 'OK' after every command?

### Boss level

Turn your panel into a .exe or .app with PyInstaller and give it to another class.

Objective check

# Can I really do it? Prove it.

| | | |
|---|---|---|
| 01 | Show your command table. What does each command do? | ○ ○ ○ |
| 02 | Show Python sending a command and reading a reply. Where is the 2-second wait? | ○ ○ ○ |
| 03 | Demonstrate your GUI controlling the LED and servo, with a live reading. | ○ ○ ○ |

**Rate yourself**

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

**Teacher:** Control room demo: each group drives another group's hardware using only the other group's panel.

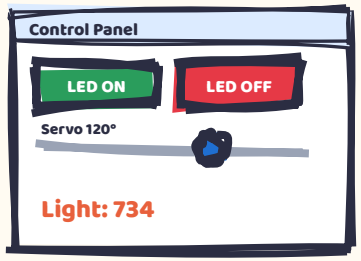
Show what you know

# What did we learn?

- [x] A protocol is an agreed set of messages
- [x] Letter + number + newline keeps commands simple
- [x] pyserial writes bytes and reads lines
- [x] tkinter apps are event-driven; mainloop waits for clicks
- [x] root.after() repeats tasks without freezing

**At home**

Look at a microwave or washing machine panel. Which buttons send which 'commands'?




**Next class:** p5.js: Flappy Voice Game!

**Exit ticket:** what would happen if Python sent 'L1' but forgot the newline?